Unit 13 Instrumentation

(1) Client-Side Vs. Server Side Instrumentation 1.6.2

(1) What is Instrumentation?

→ Instrumentation refers to tracking what user do - clicks, scrolls, purchases, etc - so that experimentors can measure and analyze behavior in A/B testing.

Client-Side Vs. Server-Side Instrumentation

(2) ☑ Server-Side Instrumentation (Back-End).

What it is:

Code runs on your company's servers (before the webpage is even shown to users). All tracking and Logging on the backend.

Example:

- A user clicks "Buy"
- The click is sent to the server.
- The server records the purchase event into logs / database

Advantages (why people use it):

- very reliable - server has full control
- No browser bugs, no ad blockers.
- Secure - not affected by client-side failures

Dis advantages

- Can miss user interface behaviors (scroll, hovers, time-on-page)
- Limited view of what the user sees or interact with visually.

(3) ☑ Client-side instrumentation (front-end).

What it is:

Code runs in the user's browser or app. Javascript or app SDK logs action after the page is loaded.

Example:

- A user hovers over a product image.
- A Javascript snippet logs in action and sends it to analytics.

Advantage:

- Can measure what's happening in the browser.
- useful for UX/UI experiment - e.g., clicks, scrolls, form interactions.
- works with 3rd-party tools like Google Analytics.

Disadvantage:

- Data loss risk; ad blockers, browser crashes, JS errors.
- users with slow networks may never load tracking code.
- less secure — attackers might spoof it.

(4) Comparison Table.

Features	Server-side	Client-side
Where it runs	Backend / server	Browser / app.
Resiliency	High	Medium (can fail silently)
Captures UX Interactions.	No	Yes
Impacted by Ad Blockers	No	Yes

Latency Sensitive	Less	Yes
Setup Complexity	Higher	Lower with Libraries

(5) Real-World Analogy

Imagine a restaurant:

- server-side = The kitchen keeps a log of every order placed (what dish, how it was prepared, time stamp)
- Client-side = The waiter observes customers: Did they frown at the menu? Did they ask questions? Did they send anything?

So:

- server-side sees confirmed orders
- client-side sees customer behavior before and during ordering.

(6) Mathematical Concept Behind This.

2 x related to: Missing Data Mechanisms in Statistics

In client-side instrumentation, some data is Missing Not at Random (MNAR)

Types of missingness

Type	Description	Related to?
MCAR	Missing completely at random	Almost never happens
MAR	Missing at random	server-side (some ad blocker cases)
MNAR	Missing not at random	client-side (ad blockers, slow loads)

When data is MNAR, eg many estimates (like conversion rate) may be biased.

| server-side gives more unbiased, complete data
| client-side may need correction technique (e.g. inverse probability weighting)

(7) Interlinked Concept.

- Unit 11 - Observational studies: Bias from unobserved data. → same problem in client-side tracking
- Unit 12 - Client-side Experiments: You're more exposed to browser-level failures, sampling biases.
- Unit 3 - Twyman's Law: "If it looks too good, it probably isn't." A skewed client-side conversion rate might mislead you.

(8) Final Takeaways.

If You care About	Use
Accuracy, Trust, Payment	Server-side
UI/UX, Button Clicks	client-side
End-to-End Tracking	☑ Both (Hybrid).

(2) "Processing Logs from Multiple Sources" Page (164)

(2) "Processing Logs from Multiple Sources" page (164)

1. What does this means?

In large-scale A/B testing system, data comes from multiple systems:

- Client-side logs (browser events, UI clicks)
- Server-side logs (backend events, purchases logins)
- 3rd party logs (payment gateways, telemetry, email opens etc)

> The challenge is: These logs have to be merged, time-aligned and cleaned to generate accurate experiment metrics.

2. Why is this hard?

Logs from different systems may:

- use different schemas or formats
- Have different timestamps (due to clock skew or time zones)
- Have missing data
- Be delayed (e.g. client logs sent hours later due to network)

so your A/B test analysis might fail or mislead if these are not handled properly.

(3) Real-World Analogy.

Restaurant Kitchen Analogy:

Imagine you're trying to understand how people enjoys meals by combining:

- Waiter notes: what people ordered, how they reacted (Client-side logs)
- kitchen system: when food was prepared and served (server-side logs)

- Billing systems : what they paid for, tip given (3rd-party logs)

Problems:

- waitress forgets to write table number (missing ID)
- kitchen uses 24-hour clock ; waitress uses 12-hour clock (timestamp mismatch).
- Billing logs came next day (delayed ingestion).

You can't understand the true customer journey unless you join and clean all logs properly.

(4) Key challenges.

1. Time skew

Different logs may record same event at different time due to:

- Clock drift
- Delayed syncing
- Log ingestion lag.

2. Different Identifiers

User might be!

- user_id in one log
- cookie_id in another
- device_id in a third

You need ID stitching (user resolution).

3. Event Duplication or Loss

Client-side logs might miss events (ad blockers), while server logs might double-count due to retries.

4. Experiment Assignment Must be Consistent.

Logs should all reflect correct treatment group - even if processed from different sources.

(5) Related Concept in the Book.

Concept	Connection
Unit 12 : Client-side Experiment	Multiple data sources increases error risk
Unit 13 : Twyman's Law	Misaligned logs can show too-good-to-be true results.
Unit 6 : Data Quality	Garbage in-garbage insights
Unit 20 : Diagnosing Result	Inconsistent logs can explain odd experiment outcomes.

(6) Mathematical Foundation

This problem links to data reconciliation, relational joins, and time-series alignment.

Think of logs as tables

server_log (user_id, timestamp, action)
client_log (cookie_id, timestamp, page_view)
Ad_log (session_id, ad_shown)

You need to:

- Convert all timestamps to one time zones.
- Join by a common key (via ID mapping)
- Handle missing joins (left / right / outer joins)
- Remove duplicates.

This is relational algebra + probabilistic identity resolution.

(7) Key Takeaways.

Concept	Summary
Multiple logs	→ Reality in modern experimentation.
Need unification	→ Must stitch, align, deduplicate
Quality risk	→ Errors here → invalid experiment conclusions.
Math concept	→ Joins, probabilities, matching, time-alignment.
Tools	→ Use pipelines (Airflow, Spark, BigQuery), versioning, validation scripts.

(3) "Culture of Instrumentation" (Page 163)

(1) What is "Culture of Instrumentation"?

It means creating a mindset and organizational habit where:

> Every feature, button, event, and change is logged deliberately — with the goal of measurement, learning and decision-making.

It's not just about having logs —
it's about valuing measurement and making it a default practice, not a afterthought.

(2) Real-world Analogy

Imagine you're managing a smart factory:

- Machine operates constantly
- If no sensors are placed, you won't know what failed
- If some sensors are there but not always checked, data is useless.

A culture of instrumentation means:

- You build sensors into every machine, by default.
- Engineers can't launch a new machine unless it has telemetry.
- Decisions are based on sensor data — not guesswork.

Just like that, in experimentations:

> If a product or feature is not ~~before~~ instrument, you can't measure it, and therefore can't improve it scientifically.

③ Why it Matters in Experimentation:

1. Unmeasured ≠ Unimportant

If you don't log a click, you can't analyze:
- Drop-off points
- Latency impact
- Unexpected bugs

2. Instrumentation should be default, Not Optional.

If teams wait until after shipping a feature to think about logging, it's often:
- Too late
- Inconsistent
- Incomplete

A strong culture ensures logging is part of the product design and dev cycle.

3. Iteration Requires Feedback

Good experimentation culture requires:
- Accurate data
- Time data
- Trusted pipelines

Instrumentation feeds all three.

④ Mathematical Foundation:

Instrumentation = Data Collection Mechanism

In experiment design, instrumentation defines your observables.

Think of in terms of function mappings

- Let U be the universe of user behaviors.
- Let $I : U \to D$ be the instrumentation function that maps behaviors to data logs.

> If I is incomplete or undefined over parts of U..
> → You lose observability → you lose statistical power.

In statistics, this is related to missing data theory, measurement error, and identifiability.

⑤ Linked Concept

Concept	How it is Connected.
Units: Data Quality	→ Culture ensures completeness, timeliness, and accuracy of logging.
Units: Observational Studies	→ If key variables aren't logged, causal inference fails
Unit 20: Diagnosing Results	→ Helps detect bugs or test mis configurations using logs.
Unit 12: Client-Side Instrumentation	→ culture ensures frontside logging is still enforced and verified.

④ Mathematical Foundation:

Instrumentation = Data Collection Mechanism

In experiment design, instrumentation defines your observables.

Think of in terms of function mappings

- Let U be the universe of user behaviors.
- Let $I : U \to D$ be the instrumentation function that maps behaviors to data logs.

> If I is incomplete or undefined over parts of U..
> → You lose observability → you lose statistical power.

In statistics, this is related to missing data theory, measurement error, and identifiability.

⑤ Linked Concept

Concept	How it is Connected.
Unit 8: Data Quality	→ Culture ensures completeness, timeliness, and accuracy of logging.
Unit 11: Observational studies	→ If key variables aren't logged, causal inference fails
Unit 20: Diagnosing Results	→ Helps detect bugs or test mis configurations using logs.
Unit 12: Client-Side Instrumentation	→ Culture ensures logging is still enforced and verified.

(6) Key Takeaways

Principle	Explanation
Instrument everything	No click, view, or interaction should go unlogged.
Make it automatic	Instrumentation should be part of the CI/CD process.
Measure what matters	Align instrumentation with key metrics.
Encourage curiosity	A culture of measurement fuels better product question and decisions.